Day 4,5,6: Git and GitHub

Sections:-

1. Why use git and github?
2. Remote and Local Concept.
3. First directory configuration- git init, git config, git clone
4. Basics(I)- git add, git commit, git status, git pull
5. Basics(II)-git branch, git checkout, git merge, git push, PR raise
6. Working flow for the Silicon_2026_repo

1) Why Do We Need GitHub?

Imagine this. You have been working on a project for the last two weeks. Every day you open VS Code, write code, fix bugs, add features, and slowly build your application. The project has now grown to more than 1000 lines of code. One morning, you turn on your laptop and realize something terrible has happened. Maybe your hard disk got corrupted, maybe you accidentally deleted the project folder, or maybe your laptop stopped working altogether. Suddenly, all your code is gone.

That would be a nightmare 😭.

This is one of the reasons developers use GitHub. But it is not the only reason.

1) Backup of Your Code

Just like we store our photos in Google Photos and our documents in Google Drive, developers store their code on GitHub.

Whenever you push your code to GitHub, a copy of your project is stored online.

So even if your laptop crashes or your project gets deleted from your machine, you can simply download the project again from GitHub.

Think of GitHub as a cloud backup for your code.

---

2) Sharing Code with Other Developers

Suppose two developers are working on the same project.

Developer A is building the Login page.

Developer B is building the Dashboard page.

If both developers create separate projects on their own computers and work independently, then at the end there will be two different projects instead of one complete project.

The developers need a common place where they can share their code with each other.

GitHub provides that common place.

Both developers can upload their code to the same repository and contribute to the same project.

---

3) Version Control (The Most Important Reason)

At first, you might think:

"Why not simply upload the project to Google Drive and let everyone download it from there?"

This sounds reasonable, but it creates a huge problem.

Suppose Developer A and Developer B both download the project.

Now:

- Developer A adds a Login page.
- Developer B adds a Dashboard page.

Both developers have modified the same project.

Now whose project should be uploaded to Google Drive?

If Developer A uploads first and Developer B uploads later, Developer B's upload may overwrite Developer A's changes.

Even worse, what if both developers edited the same file?

Now nobody knows:

- What changed?
- Who changed it?

- Which version is the latest?
- Which version should be kept?
- How should the changes be combined?

This is where Git comes in.

Git is a version control system.

Git keeps track of every change made to the project.

Whenever you make changes, Git records:

- What changed
- Who changed it
- When it was changed

If two developers modify different parts of the project, Git can merge their work automatically.

If two developers modify the same code, Git detects the conflict and asks the developers to resolve it.

Git also allows you to go back to older versions of your project. So if you accidentally break something today, you can return to a working version from yesterday.

In simple words:

- GitHub is the place where code is stored and shared.
- Git is the software that tracks changes and manages different versions of the code.

2) **Remote and Local concept:-**

Before learning Git commands, you first need to understand what a repository is.

Suppose you create a folder called:

Attendance-project

and start writing code inside it. Right now, it is just a normal folder. Git does not know anything about it.

Now suppose you run:

git init

inside that folder.

Something special happens. Git starts tracking the folder and all the files inside it. A folder that Git tracks is called a **repository** (or simply **repo**).

So you can think of a repository as:

> A folder whose changes are being tracked by Git.

Local Repository

Now imagine a company starts a new project.

They create a repository and upload it to GitHub.

Suppose the repository contains:

Login Page
Dashboard
Profile Page
Navbar

You join the company as a developer and are asked to work on this project.

You obviously cannot work directly on the company's copy. If ten developers started editing the same files at the same time, chaos would follow 😅.

So instead, you create your own copy of the repository on your computer using:

git clone

Now the entire project exists on your laptop.

This copy is called the **local repository**.

The local repository is simply your personal copy of the project where you can safely make changes.

Remote Repository

The repository stored on GitHub is called the **remote repository**.

Think of it as the central copy of the project that everyone in the team shares.

A company usually creates and maintains this repository.

All developers connect to the same remote repository.

You can think of the remote repository as the "official" version of the project.

3) First-Time Repository Setup

Before you start working with Git, there are three important commands you should know: `git init`, `git config`, and `git clone`.

1) git init

Suppose you have created a project folder and started writing code inside it. Right now, it is just a normal folder and Git is not tracking it.

To tell Git to start tracking the folder, use:

```
git init
```

This converts the folder into a Git repository. From this point onwards, Git can track changes made to the files inside the folder.

Think of `git init` as:

> "Git, please start monitoring this folder."

You usually use `git init` when creating a brand-new project from scratch.

---

2) git config

Whenever you create commits, Git stores information about who made those changes.

Before using Git for the first time, you should configure your name and email:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Git will attach this information to your commits.

For example, if three developers are working on the same project, Git can identify who made which changes.

You only need to do this once on your machine.

Think of `git config` as:

"Git, this is who I am."

---

3) git clone

Most of the time, you are not creating a project from scratch. Instead, a project already exists on GitHub and you want a copy of it on your computer.

For this, use:

```
git clone repository-url
```

Git downloads the entire repository from GitHub and creates a local copy on your machine.

For example:

```
git clone https://github.com/company/project.git
```

After cloning, you have:

- All project files
- Complete Git history
- Connection to the remote repository

Think of `git clone` as:

"Give me my own copy of this GitHub repository."

When Should You Use Which Command?

Use `git init` when:

- You have created a new project from scratch.
- The folder is not yet a Git repository.

Use `git config` when:

- You are setting up Git on your machine for the first time.

Use `git clone` when:

- A repository already exists on GitHub.
- You want your own local copy of that repository.

In real companies, `git clone` is used far more often than `git init` because the project usually already exists and developers simply clone it to start working.

4)Basics(I)- git add, git commit, git status, git pull

After you have cloned a project, these are the first Git commands you should become comfortable with. Together, they form the basic workflow that developers follow every day.

The typical workflow is:

git pull

↓

Make Changes

↓

git status

↓

git add

↓

git commit

↓

git push

(We will discuss `git push` in detail later.)

---

1) git pull

Command:

git pull

*Purpose:

Downloads the latest changes from the remote repository and updates your local repository.

Before starting work, you should usually run `git pull` to make sure you have the latest version of the project.

*Steps:

1. Open the project repository.
2. Run `git pull`.
3. Git downloads any new changes from GitHub.
4. Your local repository is now up to date.

*Example:

git pull

---

2) Make Your Changes

After pulling the latest version of the project, start working on your assigned task.

This is the step where you actually write code.

You might:

- Create a new component.
- Fix a bug.
- Update an existing page.
- Add a new feature.
- Modify HTML, CSS, or TypeScript files.

At this point, the changes exist only on your machine. Git has noticed that files have changed, but it has not yet recorded those changes anywhere.

The next step is to check what Git has detected.

---

3) git status

*Command:

git status

*Purpose:

Shows the current state of your repository.

It tells you:

- Which files have been modified.
- Which files are staged.
- Which files are untracked.
- Which branch you are currently on.

*Steps:

1. Run `git status`.
2. Observe the files that Git reports as changed.

3. Verify that the files you intended to modify are listed.

*Example:

git status

This is one of the most commonly used Git commands because it helps you understand what is happening in your repository before performing any other operation.

---

4) git add

*Command:

git add file-name

or

git add .

*Purpose:

Moves changes from the Working Directory to the Staging Area. Git does not automatically include modified files in a commit. You must explicitly tell Git which changes should be committed.

*Steps:

1. Review the changes using `git status`.
2. Stage the required files using `git add`.
3. Run `git status` again.
4. Verify that the files are now staged.

*Examples:

git add app.component.ts
git add .

The dot (`.`) means "stage all modified files."

---

5) git commit

*Command:

git commit -m "commit message"

*Purpose:

Creates a snapshot of all staged changes.

A commit is a saved version of your project at a particular point in time.

*Steps:

1. Stage your changes using `git add`.
2. Run `git commit`.
3. Write a meaningful commit message.
4. Git creates a new commit in your local repository.

*Example:

git commit -m "Added login functionality"

-Good commit messages:

-Added attendance page

-Implemented student search

-Fixed navbar styling issue

-Bad commit messages:

-Changes
-Update
-Fixed stuff

The commit message should clearly describe what was changed.

5)Basics(II)-git branch, git checkout, git merge, git push, PR raise

In a company, every developer usually has access to the main repository (the remote repository containing the actual project).

Now imagine Developer A directly writes code in the `main` branch and accidentally introduces a bug.

The next time other developers run:

git pull

they will receive that incorrect code as well.

Since the `main` branch is usually considered the stable version of the project, developers avoid making direct changes to it.

Instead, they create separate branches, make their changes there, and request that those changes be added to the main branch only after review.

This is done using a Pull Request (PR).

The typical workflow is:

git checkout main

↓

git pull

↓

git branch new-branch

↓

git checkout new-branch

↓

Make Changes

↓

git add

↓

git commit

↓

git push

↓

Raise PR

↓

Code Review

↓

git merge

---

1) git branch

*Command: git branch branch-name

*Purpose: Creates a new branch.

A branch is an isolated workspace where you can make changes without affecting the main branch.

*Steps:

1. Ensure your main branch is up to date.
2. Create a new branch.
3. Git creates the branch but does not switch to it automatically.

*Example: git branch login-feature

---

2) git checkout

*Command: git checkout branch-name

*Purpose: Switches from one branch to another.

*Steps:

1. Create a branch using `git branch`.
2. Switch to the branch using `git checkout`.
3. Verify the branch using `git status`.

*Example: git checkout login-feature

*A commonly used shortcut is: git checkout -b login-feature

This creates the branch and switches to it in a single command.

---

3) Make Your Changes

After switching to your branch:

- Create components.
- Fix bugs.
- Add features.
- Modify existing code.

Since you are working in your own branch, the main branch remains unaffected.

---

4) git add

Stage the files you want to commit.

* Example: git add .

---

5) git commit

Create a snapshot of the staged changes.

*Example: git commit -m "Added login page"

---

6) git push

*Command: git push origin branch-name

*Purpose: Uploads your branch and commits to the remote repository.

*Steps:

1. Commit your changes.
2. Push the branch.
3. GitHub now contains a copy of your branch.

*Example: git push origin login-feature

Notice that you are pushing your branch, not the `main` branch.

---

7) Raise a Pull Request (PR)

*Purpose: Requests that your branch be merged into another branch (usually `main`).

Think of a PR as saying:

> "I have completed my work. Please review it and, if everything looks good, add it to the main project."

*Steps:

1. Open the repository on GitHub.
2. Select your branch.
3. Click "Create Pull Request".
4. Add a title and description.
5. Submit the PR.

The PR is then reviewed by senior developers, team leads, or other team members.

---

8) Code Review

Before merging, reviewers typically check:

- Is the code working correctly?
- Is the coding style correct?
- Are there any bugs?
- Can the implementation be improved?

The reviewer may:

- Approve the PR.

- Request changes.
- Reject the PR.

---

9) git merge

*Command: git merge branch-name

*Purpose: Combines changes from one branch into another.

After a PR is approved, the feature branch is merged into `main`.

*Example: git merge login-feature

After merging:

main
├── Existing Features
└── Login Feature

The changes are now part of the official project.

Quick Summary

Command	Purpose
`git branch`	Create a new branch
`git checkout`	Switch to another branch
`git push`	Upload local commits to GitHub
Pull Request (PR)	Request code review and merging
`git merge`	Combine one branch with another

6)Working Flow for the Silicon_2026 Repository:-

Clone the Repository (only once)

↓

git checkout main

↓

git pull

↓

git branch new-branch

↓

git checkout new-branch

↓

Make Changes

↓

git add

↓

git commit

↓

git push origin new-branch

↓

Raise Pull Request (PR)

↓

Code Review

↓

Merge into Main